Trader Copilot

System Documentation

Phase 3 — Complete System | Version 3.0 | Confidential

This document describes the complete architecture, logic, and operation of Trader Copilot — a proprietary algorithmic trading alert system built on Smart Money Concepts (SMC) and ICT methodology. It covers every component from live data ingestion through pattern detection, signal generation, ML-enhanced confidence scoring, journalling, and alert delivery.

Authored by Ntando Miya and Sbonelo. All logic, entry models, and rule definitions reflect proprietary trading methodology developed and refined through live market analysis.

Contents

13. Roadmap

1. System Overview

Trader Copilot is a fully automated, rules-based SMC alert engine enhanced with a machine learning confidence layer. It monitors multiple currency pairs and instruments simultaneously, detects high-quality trade setups based on institutional order flow logic, scores each setup for confluence, filters weak signals using a trained ML model, and delivers real-time alerts via console output and webhook.

What the system does

At its core, Trader Copilot watches the market the same way a skilled SMC trader would — identifying liquidity, structure breaks, order blocks, and fair value gaps — but does so across multiple pairs simultaneously and without fatigue. Every 30 minutes it pulls fresh candle data from MetaTrader 5, runs the full analysis pipeline on each configured pair, and fires an alert only when a high-probability setup is confirmed.

What it is not

Trader Copilot is an alert system, not an execution bot. It identifies and notifies — it does not place trades automatically. The trader retains full discretion over which alerts to act on.

Key capabilities

- **Multi-pair scanning:** Monitors up to 9 pairs simultaneously in a single process — EURUSD, GBPUSD, EURAUD, EURCAD, AUDCAD, CADJPY, GBPCAD, XAUUSD, NAS100.
- **Two entry pipelines:** Currency pairs use the Breakout & Retest model on 30M. XAUUSD and NAS100 use the Unicorn Entry Model on 4H/15M.
- **SMC rules-based detection:** Every signal requires a confirmed structure break, order block, and retest. Wick-only breaks are rejected.
- **ML confidence layer:** A trained classifier scores each rules-confirmed signal with a win probability. Signals below 45% are filtered out.
- **Trade journal:** Every signal is logged to a SQLite database. Outcomes are recorded, enabling win rate and performance analysis.
- **Backtester:** Bar-by-bar walk-forward replay against historical MT5 CSV data — no lookahead bias.
- **Duplicate prevention:** The same setup cannot fire repeatedly across cycles. Active alerts expire after 5 cycles (~2.5 hours).
- **Webhook delivery:** Alerts are dispatched to Telegram or Discord in real time.

2. Architecture

2.1 Component map

The system is composed of eight primary components. Each has a single responsibility and communicates through well-defined interfaces.

Component	File	Responsibility
MT5 Connector	utils/mt5_connector.py	Connects to MetaTrader 5, fetches OHLCV candles per pair and timeframe.
Multi-pair runner	run_multi.py	Poll loop. Routes each pair to the correct pipeline, manages duplicate prevention, dispatches results.
TraderCopilot engine	engine.py	Orchestrator. Calls structure, pattern, POI, and signal components in sequence. Exposes analyse() and analyse_currency_30m().
Structure engine	core/structure_engine.py	Detects swing highs/lows, determines bias (bullish/bearish/ranging), identifies BOS and CHoCH.
Pattern engine	patterns/pattern_engine.py	Detects Double Top/Bottom, Head & Shoulders, Flag patterns, and the Breakout & Retest model.
POI engine	core/poi_engine.py	Detects Fair Value Gaps and Order Blocks. Confirms retest of identified zones.
Signal generator	core/signal_generator.py	Scores confluence (1-5), builds TradeSignal with entry, SL, TP, R:R.
ML confidence layer	ml/ml_engine.py	Wraps the rules engine output. Scores each signal with win_probability and confidence_tier. Filters below threshold.
Trade journal	journal/trade_journal.py	SQLite database. Logs every signal and outcome. Produces performance statistics.
Backtester	backtest/backtester.py	Walk-forward bar-by-bar simulation against historical CSVs.
Alert engine	alerts/alert_engine.py	Formats and dispatches confirmed signals and pending setup notifications.
Pair config	config/pairs.py	Per-instrument settings — timeframes, pip size, killzone windows, tolerances.

2.2 Data flow

The pipeline is strictly sequential and stateless between cycles. Each 30-minute poll triggers a fresh analysis from scratch — no state is carried between runs except the duplicate alert tracker and the

persistent journal database.

Step	Component	Action
1	MT5 Connector	Fetches 300 candles per pair at the required timeframe(s).
2	Pair router	Determines whether to use the currency 30M pipeline or the HTF/LTF pipeline.
3	Structure engine	Detects swings, determines bias. Ranging bias exits early — no signal.
4	Pattern engine	Scans for valid pattern (Double Top, H&S;, Flag, Breakout & Retest). No pattern exits early.
5	POI engine	Detects OB and FVG at the pattern's key level. Confirms retest has occurred.
6	Signal generator	Scores confluence. Builds TradeSignal with all fields populated.
7	ML engine	Scores signal with win_probability. Filters if below 45% threshold.
8	Duplicate check	Suppresses alert if same symbol+direction+entry already active.
9	Alert engine	Prints formatted alert to console. Logs to JSONL file. Sends to webhook.
10	Trade journal	Logs signal to SQLite for later outcome recording and analysis.

3. Data Ingestion — MT5 Connector

The MT5 connector is the system's only external dependency. It connects to a running MetaTrader 5 terminal on the local machine and fetches OHLCV candlestick data for any configured pair and timeframe.

How it works

- The MetaTrader5 Python package communicates with the MT5 terminal via a local socket connection.
- MT5 must be open, logged in to a live or demo account, and have algorithmic trading enabled.
- The connector requests the last N candles for a given symbol and timeframe (e.g. 300 candles of EURUSD 30M).
- Candles are returned as a list of Candle objects with timestamp, open, high, low, close, and volume.
- If MT5 is not running or returns no data, the connector logs a warning and the cycle skips that pair.

Supported timeframes

The connector supports all standard MT5 timeframes: M1, M5, M15, M30, H1, H4, D1. Currency pairs use M30 (30 minutes). XAUUSD and NAS100 use H4 for structure and M15 for entry.

4. Multi-Pair Runner

run_multi.py is the entry point for live operation. It runs a continuous poll loop that scans all configured pairs in a single process on a shared heartbeat.

Pair routing

The runner automatically routes each pair to the correct pipeline based on its symbol:

Pairs	Pipeline	Method
EURUSD, GBPUSD, EURAUD, EURCAD, AUDCAD, CADJPY, GBPCAD	Currency pipeline	analyse_currency_30m() — Breakout & Retest model on 30M
XAUUSD, NAS100	Commodity pipeline	analyse() — Unicorn Entry Model on HTF/LTF

Duplicate alert prevention

The AlertTracker class prevents the same setup from firing repeatedly across multiple cycles. A setup is uniquely identified by its symbol, direction, and entry price (rounded to 4 decimal places). Once an alert fires, it is registered with a TTL of 5 cycles — approximately 2.5 hours at 30-minute intervals. The same setup will not alert again until the TTL expires.

Usage

```
python run_multi.py # All pairs
python run_multi.py --pairs EURUSD EURAUD # Specific pairs
python run_multi.py --webhook https://... # With webhook
python run_multi.py --interval 30 # Custom interval
```

5. Rules Engine

The rules engine is the analytical core of the system. It consists of four sub-components that run in sequence: structure detection, pattern recognition, point of interest identification, and signal generation. Every step must produce a valid result for a signal to be generated — any failure exits the pipeline early.

5.1 Structure Engine

The structure engine analyses candle data to identify the market's directional bias and key structural levels. It is the first step in both pipelines.

Swing detection

Swing highs and lows are identified using a left/right pivot algorithm. A candle is a swing high if its high is the highest within N candles on each side. The default is 3 candles left and 3 right for HTF, 2 and 2 for LTF entry timeframes.

Bias determination

Bias is determined from the sequence of swing highs and lows. A series of higher highs and higher lows produces a bullish bias. Lower highs and lower lows produce a bearish bias. Mixed sequences produce a ranging bias, which causes the pipeline to exit immediately — ranging markets do not generate signals.

Structure labels

Label	Name	Meaning
HH	Higher High	Bullish continuation — new high above previous high
HL	Higher Low	Bullish continuation — new low above previous low
LH	Lower High	Bearish continuation — new high below previous high
LL	Lower Low	Bearish continuation — new low below previous low
EQH	Equal High	Liquidity level — stop hunts common above
EQL	Equal Low	Liquidity level — stop hunts common below

5.2 Pattern Engine

The pattern engine scans for the four supported trade setup patterns. Each pattern has a precise definition based on Ntando's proprietary methodology. The core BOS rule is consistent across all patterns: the displacement candle's body must close beyond the structural level. Wick-only breaks are explicitly rejected.

Double Top (Bearish)

Two approximately equal swing highs (within peak_equality_pips). Second high sweeps above the first (stop hunt). Neckline = swing low between the two peaks. Displacement candle body closes below neckline, leaving FVG at neckline. Entry on retest of neckline FVG.

Double Bottom (Bullish)

Mirror of Double Top. Two equal lows, second sweeps below first. Body closes above neckline. FVG at neckline is the entry zone.

Head & Shoulders (Bearish)

Three swing highs where the middle (head) is the highest. Left and right shoulders approximately equal. Neckline = average of the two lows between shoulders and head. Body close below neckline triggers signal.

Inverse H&S; (Bullish)

Mirror of H&S.; Three swing lows, middle is lowest. Body close above neckline triggers signal.

Descending Flag (Bearish)

Consolidation range within recent candles narrower than average range (momentum loss). Body closes below consolidation low. FVG at breakout is entry zone.

Ascending Flag (Bullish)

Mirror of Descending Flag. Body closes above consolidation high.

Breakout & Retest (Both)

Currency-specific model (30M). Channel defines bias. Corrective wedge inside channel. Body close beyond wedge boundary. OB left by breakout candle. Entry at top/bottom of OB on retest. Covered in detail in Section 6.1.

5.3 POI Engine

The Point of Interest engine identifies the precise entry zones within a confirmed pattern. It detects two types of POI: Fair Value Gaps and Order Blocks.

Fair Value Gap (FVG)

A Fair Value Gap is a three-candle imbalance where the displacement candle leaves a gap between the previous candle's low and the next candle's high (for bearish) or the previous high and next low (for bullish). The gap must be at least fvg_min_pips in size. The FVG must be located near the structural neckline level to qualify as a valid entry zone.

Order Block (OB)

An Order Block is the last candle of the opposite colour immediately before the displacement move. For a bearish OB, it is the last bullish candle before the sweep rejection. The OB represents the zone where institutional orders were placed. In the Breakout & Retest model, the entry is specifically at the top of the OB (for shorts) or the bottom of the OB (for longs).

Retest detection

After the displacement, the POI engine monitors subsequent candles for a return to the identified zone. For bearish trades, the retest occurs when a candle's high reaches within tolerance of the neckline or OB top. If an FVG is present, the retest must also enter the FVG zone to receive the maximum confirmation signal.

5.4 Signal Generator

The signal generator is the final step of the rules engine. It receives a confirmed pattern, entry candle, sweep, and OB, calculates the trade levels, scores the confluence, and builds a TradeSignal object that flows into the ML layer and alert engine.

Confluence scoring (1–5)

Each confirmed signal is scored on five criteria:

Points	Criterion	Condition
+1	Base	Pattern confirmed with valid BOS (body close rule satisfied)
+1	Retest confirmed	Price has returned to the OB/FVG zone after displacement
+1	FVG present	A valid Fair Value Gap exists at the entry zone
+1	Killzone active	Current time falls within the pair's active trading session
+1	R:R >= 2.0	Minimum 1:2 risk-to-reward achieved at the target level

6. Entry Models

Trader Copilot implements two distinct entry models, each optimised for a different instrument type and timeframe structure.

6.1 Breakout & Retest — Currency Pairs (30M)

This model was developed specifically for currency pairs through extensive chart study across EURUSD, GBPUSD, EURAUD, EURCAD, AUDCAD, CADJPY, and GBPCAD. It operates exclusively on the 30-minute timeframe.

The setup sequence

1. Channel

A descending or ascending channel is identified on the 30M chart. The channel defines the dominant trend direction (bearish or bullish). The 90% rule level marks the origin of the channel — this is the hard invalidation boundary for the trade.

2. Corrective wedge

Inside the channel, price forms a corrective structure against the trend — a rising wedge in a downtrend, or a falling wedge in an uptrend. The wedge is characterised by a tightening range (converging trendlines) and slowing momentum. This represents institutional absorption before the next impulse.

3. Breakout

Price breaks out of the wedge in the direction of the channel trend. The breakout is only confirmed when a candle body closes beyond the wedge boundary — wick-only penetrations are rejected. The breakout candle leaves an Order Block (the last candle of opposite colour before the breakout) and optionally an FVG from the displacement.

4. Retest

After the breakout, price retraces back toward the OB. When price touches the top of the OB (for shorts) or the bottom of the OB (for longs), the entry triggers. If price closes beyond the 90% rule level before the retest occurs, the setup is invalidated.

Entry, stop, and target

Entry	Limit order at the TOP of the OB (short) or BOTTOM of the OB (long)
Stop Loss	Beyond the OB — above OB high for shorts, below OB low for longs
Hard invalidation	Price closes beyond the 90% rule level before retest

Target	Opposite channel boundary or higher timeframe supply/demand zone

6.2 Unicorn Entry Model — XAUUSD and NAS100

The Unicorn Entry Model is used for Gold and NAS100. It operates on a two-timeframe structure: the 4H defines the bias and pattern, while the 15M provides the precise entry confirmation.

The setup sequence

HTF bias (4H)

Structure engine identifies the trend direction. A bearish bias requires a sequence of LH and LL. Bullish requires HH and HL.

Pattern confirmation (4H)

One of the four supported patterns must form at the end of the trend: Double Top/Bottom, Head & Shoulders, or Flag. The neckline is the key structural level.

Liquidity sweep (15M)

Price sweeps the liquidity beyond the HTF swing high/low (stop hunt). The wick must extend beyond the level; the body must reject.

Displacement (15M)

A strong impulse candle breaks structure. The candle body must close beyond the neckline.

FVG at neckline (15M)

The displacement candle leaves a Fair Value Gap. The FVG must overlap with the neckline level to qualify.

iCHoCH retest (15M)

Price retraces into the FVG/OB zone. Entry triggers on confirmation of the retest.

7. ML Confidence Layer

Phase 3 adds a machine learning layer that wraps the rules engine output. Every signal that passes the rules engine is scored by the ML model before being dispatched. The model learns from the system's own historical trading outcomes stored in the journal.

How it works

- The ML engine receives a TradeSignal that has already passed all rules-based checks.
- It extracts a feature vector from the signal: confluence score, FVG presence, OB presence, killzone, R:R ratio, pattern type, timeframe, and session.
- The trained classifier outputs a win_probability (0.0 to 1.0) and a confidence_tier (PREMIUM / HIGH / MEDIUM / LOW).
- Signals with win_probability below the configured threshold (default 45%) are filtered out — no alert is dispatched.
- If no model has been trained yet (insufficient journal data), the ML engine falls back to the rules-based confluence score.

Training

The model is trained on closed trades from the journal where outcomes have been recorded (TP hit, SL hit, or manual close). Training requires a minimum of approximately 30 closed trades to produce meaningful predictions. The trainer is run manually:

```
python -m trader_copilot.ml.trainer
```

The model is automatically retrained daily at system startup if new journal data is available. As the journal grows, model accuracy improves. Early in the system's life the rules-based fallback handles all scoring.

Confidence tiers

Tier	Probability	Guidance
PREMIUM	0.70+	Highest confidence. All rules met, strong ML agreement.
HIGH	0.60–0.69	Strong signal. Act on these with standard risk sizing.
MEDIUM	0.50–0.59	Acceptable. Consider reduced position size.
LOW	0.45–0.49	Marginal. Borderline — discretion advised.
FILTERED	< 0.45	Signal suppressed. Alert not dispatched.

8. Trade Journal

Every signal dispatched by the system is automatically logged to a SQLite database. The journal is the system's memory — it records what was signalled, and later what the outcome was. This data feeds the ML trainer and provides performance analytics.

What is logged

Field	Description
symbol	Trading pair — e.g. EURUSD
direction	BULLISH or BEARISH
pattern	Pattern name that triggered the signal
entry_price	The limit entry price
stop_loss	The stop loss level
take_profit	The take profit target
risk_reward	Calculated R:R ratio
confluence_score	Rules-based score (1–5)
win_probability	ML model output (0.0–1.0)
fvg_present	Whether a valid FVG was detected
ob_present	Whether an OB was detected
killzone_active	Whether a killzone was active at signal time
timeframe	The entry timeframe
timestamp	UTC datetime of the signal
outcome	Recorded after trade closes: TP_HIT, SL_HIT, or MANUAL_CLOSE
pnl_r	Outcome in R-multiples (e.g. +2.4 for a full TP)

Performance statistics

The journal produces the following analytics on demand:

- Overall win rate and average R-multiple across all closed trades.
- Win rate broken down by pattern type — identifies which setups perform best.
- Win rate broken down by confluence score — validates whether higher scores predict better outcomes.
- FVG boost validation — compares win rate of signals with vs without FVG present.
- Performance by session/killzone — identifies optimal trading windows.

- Drawdown analysis — longest losing streak, maximum drawdown in R.

9. Backtester

The backtester allows the system's rules to be tested against historical data before committing to live operation. It uses a bar-by-bar walk-forward approach — the engine sees only candles up to the current bar, exactly as it would in live operation. There is no lookahead bias.

How it works

- Historical OHLCV data is exported from MT5 as a CSV file.
- The backtester iterates through the data one candle at a time, calling the engine's analysis pipeline on the candle window available up to that point.
- When a signal is generated, the backtester simulates the trade outcome by scanning forward through subsequent candles to determine whether TP or SL was hit first.
- All simulated trades are logged with entry, exit, outcome, and R-multiple.
- A summary report is produced covering win rate, profit factor, average R, maximum drawdown, best and worst trade, and breakdown by pattern and confluence score.

Backtester results should be treated as indicative, not definitive. Slippage, spread, and execution timing in live trading will differ from the idealised conditions in the simulation.

10. Alert Engine

The alert engine is responsible for formatting and delivering signals to the trader. It supports three output channels simultaneously.

Output channels

Channel	Description
Console	Formatted alert printed to the terminal window where the bot is running.
Log file	Every signal appended to a JSONL file (one JSON object per line) named {SYMBOL}_alerts.jsonl. Pending setups go to {SYMBOL}_alerts_pending.jsonl.
Webhook	Alert text POSTed to a Telegram bot or Discord webhook URL. Delivers real-time notifications to the trader's phone.

Alert types

Confirmed signal

A confirmed signal alert fires when all rules have been satisfied and the ML model has approved the signal. It includes: pair, direction, pattern, entry price, stop loss, take profit, R:R, confluence score, ML confidence tier, FVG/OB flags, killzone status, and notes.

Pending setup

A pending alert fires when a valid Breakout & Retest pattern is confirmed but the retest has not yet occurred. It notifies the trader to watch a specific price level for the limit order. When price subsequently retests that level, the confirmed signal alert fires.

11. Pair Configuration

Each instrument has a dedicated configuration in config/pairs.py. The configuration controls detection sensitivity and session filtering.

Parameter	Description
structure_tf	The higher timeframe used for structure and pattern detection.
entry_tf	The lower timeframe used for entry confirmation (sweep, FVG, retest).
killzones	Named trading sessions during which signals are considered active.
sweep_wick_pips	Minimum wick size beyond a level to qualify as a liquidity sweep.
fvg_min_pips	Minimum size of a Fair Value Gap to be considered valid.
peak_equality_pips	Maximum price difference between two peaks to be considered equal (Double Top/Bottom).
pip_size	The pip value for this instrument (0.0001 for most FX, 0.01 for JPY pairs, 1.0 for Gold and indices).

Configured instruments

Symbol	Structure TF	Entry TF	Killzones
EURUSD	30M	5M	London, New York
GBPUSD	30M	5M	London, New York
EURAUD	30M	5M	London, Sydney
EURCAD	30M	5M	London, New York
AUDCAD	30M	5M	London, Sydney, New York
CADJPY	30M	5M	London, New York, Tokyo
GBPCAD	30M	5M	London, New York
XAUUSD	4H	15M	London Open, New York Open
NAS100	4H	15M	New York Open

12. Running the System

Prerequisites

- Python 3.10 or later installed with MetaTrader5 package (pip install MetaTrader5).
- MetaTrader 5 terminal open and logged in with algorithmic trading enabled.
- Optional: Telegram or Discord webhook URL configured for mobile alerts.

Start commands

```
python run_multi.py # All pairs, 30-min interval
python run_multi.py --pairs EURUSD EURAUD CADJPY # Selected pairs only
python run_multi.py --webhook https://your-url # With webhook
python run_live.py --symbol XAUUSD --interval 15 # Single pair runner
```

Expected console output

```
2025-01-08 08:30:00 [INFO] Cycle 1 | 08:30 UTC | Scanning 9 pairs
2025-01-08 08:30:01 [INFO] EURUSD -- No setup this cycle
2025-01-08 08:30:02 [INFO] EURAUD -- SIGNAL SELL | Entry: 1.76420 | Score: 4/5
2025-01-08 08:30:03 [INFO] CADJPY -- No setup this cycle
2025-01-08 08:30:09 [INFO] Cycle 1 complete | 1 signal(s) fired | Next scan in 30
minutes
```

Stopping

Press Ctrl + C in the terminal. The runner stops cleanly and disconnects from MT5.

13. Roadmap

The following capabilities are planned for future phases of development.

Feature	Description
Web dashboard	Browser-based UI showing live alerts, journal statistics, pair controls, and ML model performance. Target users: Sbonelo and future subscribers.
SaaS multi-user	Per-user configuration, journal isolation, and billing. Each subscriber manages their own pair list and webhook.
Additional pairs	Expand coverage to GBPJPY, USDJPY, AUDUSD, and GBPAUD with pair-specific tuning.
NAS100 equal highs edge	Dedicated detection rule for the NAS100 equal highs liquidity pattern identified in live market study.
Pre-entry countdown	Alert the trader N candles before the expected retest zone, giving time to prepare a limit order.
Outcome auto-detection	Automatically detect TP/SL hits from MT5 trade history and update the journal without manual input.
Mobile app	Native iOS/Android app replacing the Telegram/Discord webhook for a branded alert delivery experience.